

Lists in Python

1. What are lists (and list documentation)

Lists documentation

Lists are one of Python's *Built-in Types*

You find detailed the documentation here among other built-in types

https:

`//python.readthedocs.io/en/latest/library/index.html`

Then choose: Sequence Types - list, tuple, range

To reach: `https://docs.python.org/3/library/stdtypes.html#sequence-types-list-tuple-range`

- Built-in Types
 - Truth Value Testing
 - Boolean Operations — and, or, not
 - Comparisons
 - Numeric Types — int, float, complex
 - Boolean Type - bool
 - Iterator Types
 - Sequence Types — list, tuple, range
 - Text and Binary Sequence Type Methods Summary
 - Text Sequence Type — str
 - Binary Sequence Types — bytes, bytearray, memoryview
 - Set Types — set, frozenset
 - Mapping Types — dict

2. Create and use a list

Create a list

```
1 cities = ["Stockholm", "Oslo", "London"]
2 temperatures = [15.5, 14.3, 17.0]
3
4 print(type(cities))
5 print(type(temperatures))
```

```
<class 'list'>
```

```
<class 'list'>
```

Concatenating lists

With '+' two lists can be concatenated.

```
1 cities_1 = ["Stockholm", "Oslo", "London"]
2 cities_2 = ["Berlin", "Paris"]
3 cities = cities_1 + cities_2
4 print(cities)
```

```
['Stockholm', 'Oslo', 'London', 'Berlin', 'Paris']
```

A new list is created, and stored in the variable `cities`.

3. Indexing and slicing lists

Indexing lists

As other *sequences* (e.g., strings),
lists can be indexed.

Important to remember:

You always start counting at 0

```
1 cities = ["Stockholm", "Oslo", "London"]  
2 print(cities[0])  
3 print(cities[2])
```

Stockholm

London

Indexing lists

One more example

Important to remember:

You always start counting at 0

```
1 tokenized_sentence = ["Hello", "there", "world", "?"]  
2 print(tokenized_sentence[3])
```

?

Indexing lists

If you provide an index that is not in the list you will get an **IndexError**.

Important to remember:

You always start counting at 0

```
1 tokenized_sentence = ["Hello", "there", "world", "?"]
2 print(tokenized_sentence[4]) # Index not in list
```

IndexError: list index out of range

Slicing lists

Lists can also be sliced.

To get ['there', 'world'], you start with the index for 'there', i.e. 1, and end with the index **after** 'world', i.e. 3

(remember that indexing starts at 0)

```
1 tokenized_sentence = ["Hello", "there", "world", "?"]  
2 print(tokenized_sentence[1:3])
```

```
['there', 'world']
```

Slicing lists

from beginning or towards the end

You can also start from the beginning or continue to the end

```
1 tokenized_sentence = ["Hello", "there", "world", "?"]
2 # From the start
3 print(tokenized_sentence[:2])
4
5 # Towards the end
6 print(tokenized_sentence[1:])
```

```
['Hello', 'there']
['there', 'world', '?']
```

Indexing lists with negative indexes

You can also use negative indexes. Then you will start counting backwards.

```
1 tokenized_sentence = ["Hello", "there", "world", "?"]  
2 print(tokenized_sentence[-1])
```

?

len()

We can make use of `len()` to avoid using an index that does not exist.

```
1 def index_safe(s, index):
2     if index < len(s):
3         return s[index]
4     else:
5         return None
6
7 tokenized_sentence = ["Hello", "there", "world", "?"]
8 print(index_safe(tokenized_sentence, 3))
9 print(index_safe(tokenized_sentence, 4))
```

?

None

4. Lists are mutable

Lists are mutable

There is an important difference between a list and a string:
Lists are mutable, and therefore they **can** be changed.

```
1 tokenized_sentence = ["Hello", "there", "world", "?"]
2 tokenized_sentence[2] = "!"
3 print(tokenized_sentence)
```

```
['Hello', 'there', '!', '??']
```

5. Looping over lists

Looping over lists

With the keywords *for* and *in* you can loop over a list

```
1 tokenized_sentence = ["Hello", "there", "world", "?"]  
2 for word in tokenized_sentence:  
3     print(word)
```

Hello there world ?

Looping over lists

Another example:

```
1 tokenized_sentence = ["Hello", "there", "world", "?"]
2 for word in tokenized_sentence:
3     print(word[0])
```

H

t

w

?

6. Checking if something is in a list

Checking if something is in a list

Also the other usage of the keyword `in` works with lists:

```
1 tokenized_sentence = ["Hello", "there", "world", "?"]
2 if 'world' in tokenized_sentence:
3     print("The word 'world' was found in the sentence")
4 else:
5     print("The word 'world' was NOT found in the sentence")
```

The word 'world' was found in the sentence

Checking if something is in a list

Another example

```
1 def is_city_in_sentence(tokenized_sentence):
2     cities = ["Stockholm", "Oslo", "London"]
3     for word in tokenized_sentence:
4         if word in cities:
5             return True
6     return False
7
8 s1 = ["Stockholm", "is", "my", "home", "town"]
9 s2 = ["I", "am", "going", "to", "Oslo"]
10 s3 = ["Hello", "there", "world", "?"]
11 print(is_city_in_sentence(s1))
12 print(is_city_in_sentence(s2))
13 print(is_city_in_sentence(s3))
```

True

True

False

7. Built-in functions applied on lists

Reflections

on the built-in functions shown so far

`print()`, `type()`, `len()` are all functions of the same type. They are examples of *built-in* functions.

Python doesn't have very many built-in functions. You find a list of them here:

<https://docs.python.org/3/library/functions.html>.

Some of them can be applied on lists.

Built-in Functions

The Python interpreter has a number of functions and types built into it that are always available listed here in alphabetical order.

Built-in Functions			
A abs() aiter() all() anext() any() ascii()	E enumerate() eval() exec()	L len() list() locals()	R range() repr() reversed() round()
B bin() bool() breakpoint() bytearray() bytes()	F filter() float() format() frozenset()	M map() max() memoryview() min()	S set() setattr() slice() sorted() staticmethod() str() sum() super()
C callable() chr() classmethod() compile() complex()	G getattr() globals()	N next()	T tuple() type()
D delattr() dict() dir() divmod()	H hasattr() hash() help() hex()	O object() oct() open() ord()	V vars()
	I id() input() int() isinstance() issubclass() iter()	P pow() print() property()	Z zip()
			_ __import__()

Reflections

on built-in functions

- ▶ `print()` takes objects to print as parameter (among other parameters), prints them and returns `None`.
- ▶ `type()` takes an object as parameter and returns its type (an instance of the `Type` class). Can be good for debugging.
- ▶ `len()` takes an object as parameter and returns its length.

More built-in functions that can be applied on lists

sorted()

sorted() can be used to sort a list. Note that a sorted **copy** of the original list is returned. (The original list stays the same.)

```
1 cities = ["Stockholm", "Oslo", "London"]
2 s_cities = sorted(cities)
3
4 print("original: " + str(cities))
5 print("s_cities: " + str(s_cities))
```

```
original: ['Stockholm', 'Oslo', 'London']
s_cities: ['London', 'Oslo', 'Stockholm']
```

Reflections

on built-in functions used on lists

Note that all examples so far have been on built-in *functions*.

- ▶ The built-in functions showcased can be applied on many different types of objects.
- ▶ They all take the lists as arguments, e.g., `len(my_list)` or `sorted([1, 3, 2])`.

Now we will go on to look at list *methods*. They are specific to list objects and are called using *dot notation*.